

Arquitectura y Especificaciones de la Plataforma IIP

Juan José Martínez Guerrero^{1,2}

1: Diseñador Industrial, 2: Ingeniero de Sistemas

<https://github.com/SpanishSyntax>

Abstract

La plataforma del Índice de Innovación Pública (IIP) constituye un ecosistema de gestión de datos de alto rendimiento, diseñado como una arquitectura de microservicios orientada al dominio (DDD) para servir como el motor central de inteligencia y trazabilidad de la Veeduría Distrital. El sistema implementa una arquitectura basada en contenedores Docker que integra componentes especializados: autenticación, lógica central (Core), persistencia (Alembic/Seeders), almacenamiento persistente en PostgreSQL y un proxy inverso Nginx. Desarrollada bajo el patrón Fast API con capas de servicios, unidades de trabajo (UOW) y repositorios, la solución centraliza el ciclo de vida completo de las versiones 2019, 2021, 2023 y 2025 del IIP, abarcando desde la gestión de cuestionarios y respuestas hasta el procesamiento analítico de resultados. Esta infraestructura está diseñada como una solución escalable y abierta, preparada para la integración futura con motores de vectorización, sistemas de Generación Aumentada por Recuperación (RAG) y protocolos de Model Context Protocol (MCP), facilitando tanto la captura de nuevas propuestas y la gestión de procesos de evaluación, como la democratización de datos a través de portales de datos abiertos.

1 Introducción

La plataforma del índice de Innovación Pública (desde ahora IIP) se erige como un sofisticado ecosistema tecnológico, concebido como un hub de datos multidominio de alta disponibilidad, diseñado específicamente para satisfacer las necesidades analíticas y de gobernanza de la Veeduría Distrital. Este sistema no solo actúa como un repositorio centralizado, sino como un motor de procesamiento inteligente capaz de orquestar la complejidad inherente a los datos distritales, integrando de manera fluida la gestión dinámica de formularios, la administración de actores estratégicos y sistemas de evaluación robustos.

Bajo una arquitectura de microservicios estrictamente desacoplada, la plataforma garantiza una separación de responsabilidades que optimiza el ciclo de vida del software, permitiendo que los servicios de autenticación, la lógica de negocio central y la capa de persistencia operen como entidades independientes, aunque perfectamente cohesionadas. Todo este desarrollo está consolidado bajo una estrategia de monorepo, la cual permite la gestión unificada del código fuente, facilitando el intercambio de lógica a través de una biblioteca interna compartida. Esta infraestructura técnica, robusta y escalable, ha sido diseñada con una visión a largo plazo, garantizando que el sistema sea capaz de evolucionar desde una solución de gestión administrativa hacia un núcleo tecnológico preparado para la integración de sistemas de vectorización, arquitecturas RAG, y protocolos de comunicación de última generación como MCP, consolidándose así como la infraestructura de datos definitiva para la innovación pública.

1.1 Propósito

El propósito fundamental de la plataforma IIP es democratizar el acceso y la gestión del conocimiento derivado del Índice de Innovación Pública (IIP), funcionando como la columna vertebral de datos para la Veeduría Distrital. El alcance del sistema abarca la consolidación histórica y analítica de todas las versiones del índice (2019, 2021, 2023 y 2025), transformando una estructura de datos fragmentada en un modelo de información coherente, versionable y auditable.

En términos operativos, la plataforma está diseñada para satisfacer tres pilares críticos:

1. Gestión Integral del Ciclo de Vida: Desde la captura de nuevas respuestas y la gestión de actores, hasta la evaluación técnica realizada por el colegio calificador.
2. Interoperabilidad de Datos: Actuar como fuente única de verdad para la publicación de datos abiertos, facilitando el consumo analítico externo y asegurando la transparencia gubernamental.
3. Extensibilidad Inteligente: Servir como base de datos para sistemas avanzados, incluyendo la futura integración de servicios de RAG (Generación Aumentada por Recuperación) y agentes de IA, permitiendo consultas semánticas complejas sobre todo el histórico de resultados del IIP.

1.2 Alcance

1.3 Audiencia

1.4 Vista panorámica

1.5 Principios de diseño

1.6 Mapa de arquitectura de alto nivel

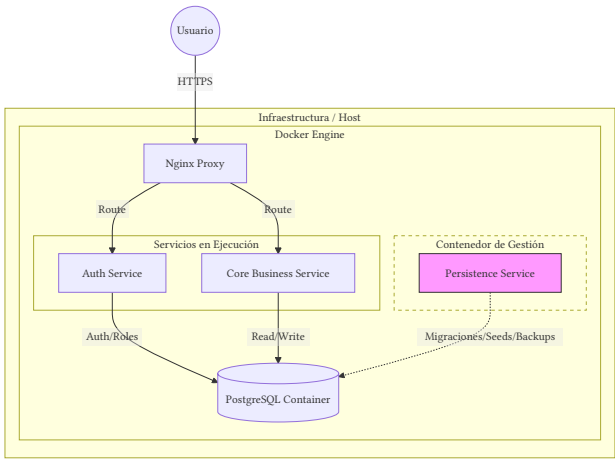


Figura 1: Arquitectura de despliegue.

El sistema se despliega mediante una arquitectura basada en contenedores Docker, orquestada para garantizar el aislamiento y la escalabilidad de cada componente. Un proxy inverso Nginx actúa como puerta de enlace, gestionando el enrutamiento del tráfico hacia los servicios correspondientes y asegurando una comunicación segura. La lógica interna sigue el patrón de diseño Domain-Driven Design (DDD), implementando una arquitectura de capas que organiza el código en servicios, unidades de trabajo (UOW) y repositorios, asegurando que la lógica de negocio permanezca desacoplada de los detalles técnicos de persistencia. Esta disposición permite una evolución independiente de cada módulo, desde la capa de autenticación hasta el motor de procesamiento de datos gestionado por el servicio de core y el motor de base de datos PostgreSQL.

2 Arquitectura de sistema

La arquitectura de la plataforma IIP se fundamenta en un diseño modular de microservicios, orquestado mediante contenedores para garantizar la independencia, la escalabilidad horizontal y el aislamiento de responsabilidades. Hemos abandonado los enfoques monolíticos tradicionales en favor de una estructura de monorepo, la cual permite la gestión unificada del código fuente bajo una estrategia de workspace. Como se ilustra en Figura 2, esta decisión estratégica facilita que los dominios funcionales (Auth, Core, Persistence) compartan una base común de utilidades y modelos (el módulo Shared), garantizando la consistencia del sistema y la reducción de la duplicidad de código sin sacrificar la independencia de despliegue de cada servicio.

Este modelo es ideal para las necesidades de la Veeduría Distrital, ya que permite una gobernanza centralizada del código mientras asegura una escalabilidad granular, facilitando ciclos de desarrollo independientes para los componentes de autenticación y lógica de negocio central.

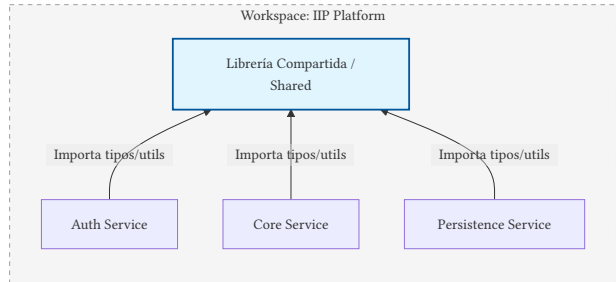


Figura 2: Dependencias de módulos dentro del monorepo.

2.1 Stack tecnológico

El sistema está construido sobre un stack moderno orientado al rendimiento y la seguridad transaccional:

- 1. **Runtime:** Python 3.12.13
- 2. **Framework:** FastAPI
- 3. **ORM & DB:** SQLAlchemy 2.0 con PostgreSQL
- 4. **Migraciones:** Alembic
- 5. **gestión de paquetes:** uv with workspace support
- 6. **Seguridad:** Argon2 para hashing y ED25519 para JWT

Para un desglose detallado de las dependencias en materia de librerías del proyecto, por favor remítase a los `pyproject.toml` correspondientes a cada contenedor y al `pyproject.toml` global del proyecto.

Servicio	Responsabilidad	Entidades Clave
Nginx	Proxy inverso y enrutamiento	Configuración, Certificados
Auth	Gestión de identidad y seguridad	User, Role, RefreshSession
Core	Lógica de negocio y procesamiento de dominio	Form, Submission, Actor, Grade
Persistence	Ciclo de vida de BD, migraciones y carga	Alembic, Seeder, Populator
Shared	Lógica común, modelos ORM y utilidades	Base, AccessContext, Hashing

Tabla 1: Componentes del sistema y sus responsabilidades.

3 Arquitectura de despliegue

3.1 Infraestructura

3.2 Arquitectura Docker

3.3 Proxy inverso

3.4 Comunicación de servicios

3.5 Variables de ambiente

3.6 Secretos y contraseñas

4 Arquitectura de dominio

4.1 Microservicios

La lógica se encuentra particionada funcionalmente. El servicio Auth gestiona exclusivamente la identidad; el servicio Core encapsula la lógica de negocio del IIP, y el servicio de Persistence actúa como el motor de estado para Alembic y los seeders, asegurando que los datos históricos de 2019, 2021, 2023 y 2025 sean inyectados y mantenidos con integridad. Todo este despliegue está protegido por una capa de Nginx, tal como se detalla en Figura 1.

4.2 DDD

El servicio Core implementa una arquitectura basada en Domain-Driven Design [1] (DDD), organizada en capas que separan las responsabilidades de presentación, aplicación, dominio e infraestructura. Esta organización permite mantener la lógica de negocio desacoplada de los mecanismos de persistencia y comunicación, facilitando la mantenibilidad, la extensibilidad y las pruebas unitarias.

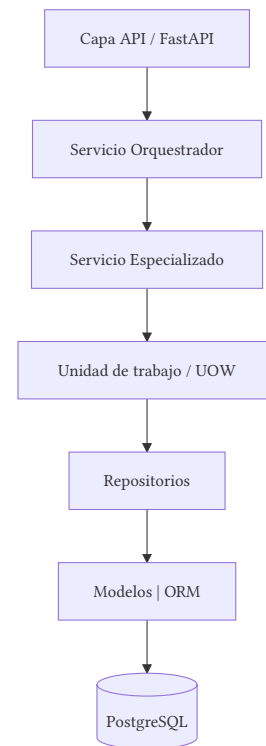


Figura 3: Arquitectura de capas orientada al dominio (DDD).

1. **Capa API:** Expone los endpoints REST mediante FastAPI y actúa como punto de entrada al sistema. Su responsabilidad se limita a la validación de solicitudes, autenticación y serialización de respuestas.
2. **Servicio Orquestrador:** Coordina el flujo de ejecución de los casos de uso. Esta capa encapsula la lógica de alto nivel, delegando las operaciones específicas en servicios especializados sin contener reglas de negocio propias.
3. **Servicios Especializados:** Implementan la lógica de negocio asociada a cada dominio funcional, como la gestión de formularios, respuestas o auditorías. Estos servicios utilizan la Unidad de Trabajo para ejecutar operaciones transaccionales de forma consistente.
4. **Unidad de Trabajo y Repositorios:** La Unidad de Trabajo (UOW) administra el ciclo de vida de las transacciones y garantiza la atomicidad de las operaciones. Los repositorios abstraen el acceso a los datos, desacoplando la lógica de negocio de la tecnología de persistencia utilizada.
5. **Modelos ORM y Base de Datos:** La capa de persistencia emplea modelos definidos con SQLAlchemy ORM para mapear las entidades del dominio a la base de datos PostgreSQL, proporcionando una interfaz orientada a objetos sobre el almacenamiento relacional.

Esta separación de responsabilidades facilita la evolución independiente de cada capa, mejora la mantenibilidad del sistema y simplifica la implementación de pruebas unitarias al reducir el acoplamiento entre la lógica de negocio y la infraestructura.

4.3 Arquitectura de capas

4.4 Servicios

4.5 Repositorios

4.6 UOW

4.7 Shared

5 Arquitectura de datos

5.1 Base de datos

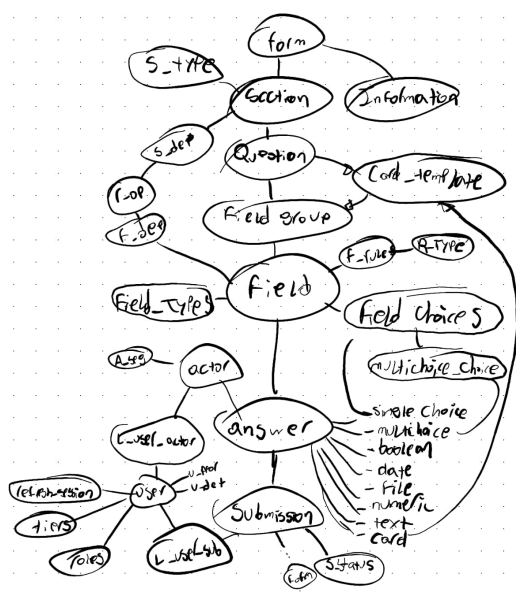


Figura 4: Arquitectura de flujo y dependencias del sistema de persistencia.

5.2 Modelos

Packages/shared/src/shared/models

- actors.py
- audit.py
- auth.py
- files.py
- forms.py
- grading.py
- __init__.py
- interactions.py
- links.py
- reference.py
- rules.py
- submissions.py
- targets.py

5.3 Relaciones

5.4 Gestión

6 Persistencia

6.1 Creación de DB schemas

Este módulo automatiza la creación de esquemas de base de datos en PostgreSQL durante el despliegue inicial del sistema, inspeccionando dinámicamente las clases del modelo de datos para garantizar la existencia de las estructuras necesarias. El script de persistencia extrae de manera única los nombres de los esquemas definidos en la clase `TargetTable` y sus clases base mediante introspección de código, identificando cada atributo que implemente el tipo `TableInfo`.

Posteriormente, abre una sesión síncrona con la base de datos y ejecuta de forma iterativa comandos seguros de creación de esquemas que previenen errores por duplicación. El flujo maneja de forma independiente cada confirmación o reversión ante fallos y genera un informe final en el registro del sistema detallando las estructuras creadas de manera exitosa y las fallidas.

El script `launcher.sh` ejecuta automáticamente este proceso de inicialización al detectar la bandera `--setup`, simplificando la configuración del entorno cuando el sistema se despliega por primera vez en un servidor virgen.

i Nota: Nota: La ejecución exitosa mediante la bandera `--setup` requiere que las credenciales y variables de entorno de la base de datos estén correctamente configuradas, y que el motor de PostgreSQL esté activo y aceptando conexiones.

6.2 Alembic

La configuración de Alembic permite gestionar el ciclo de vida de la base de datos, incluyendo la carga dinámica de modelos y la definición de una tabla personalizada para las migraciones, `alembic_automatic_version`. Para asegurar la persistencia y sincronización entre el host y el contenedor `app_persist`, el volumen de versiones de Alembic debe montarse en docker-compose, reflejando la estructura necesaria de archivos en `Persistence/src/migrator/alembic/versions`.

```
persist:
...
volumes:
  # Huesped : Contenedor
  - "/Persistence/src/migrator/alembic/versions:/api/migrator/alembic/versions"
  # Cambie sólo la ruta en huésped, NO cambie ruta en contenedor.
```

El montaje del volumen de Alembic es obligatorio, y si el directorio `versions` en el host está vacío o no coincide

con el estado actual de los modelos al levantar el servicio persistir, las migraciones automáticas fallarán al no poder contrastar la metadata con el historial de revisiones previo.

6.3 Poblado de sistema (seeds)

Este módulo automatiza la carga de datos maestros e iniciales (seeds) en la base de datos tras asegurar la existencia de los esquemas y aplicar las migraciones correspondientes. El mecanismo implementado en `seeder.py` escanea dinámicamente el directorio interno `seeds/`, ordenando alfabéticamente los archivos encontrados para garantizar una secuencia de ejecución predecible y respetar las dependencias relacionales subyacentes. Utilizando el módulo `importlib.util` de Python, el script realiza una carga reflexiva de cada archivo `.py`, busca de forma explícita una función ejecutable llamada `upgrade()` y la invoca de manera aislada dentro de un bloque controlado de excepciones. El flujo captura errores individuales por archivo para evitar que un fallo en un set de datos interrumpa todo el proceso de inicialización, generando un registro detallado en el `logger` y volcando la traza completa (`traceback`) en la consola ante cualquier eventualidad.

Al igual que los módulos previos de persistencia, este componente es invocado automáticamente por el script `launcher.sh` al ejecutar la bandera `--setup` durante el despliegue en un entorno virgen, asegurando que las tablas base queden completamente pobladas.

Para asegurar el correcto orden de inserción (por ejemplo, registrar roles antes que usuarios), el directorio debe mantener una nomenclatura secuencial estricta tal como se ilustra en la siguiente estructura física:

```
Persistence/src/seeder
├── seeder.py
├── seeds
│   ├── 00a_seed_log_action_types.py
│   ├── 00b_seed_file_types.py
│   ├── 00c_seed_roles.py
│   ├── 00d_seed_user_tiers.py
│   ├── 10a_seed_users.py
│   ├── 1b_seed_rule_types.py
│   ├── 1c_seed_relational_operators.py
│   ├── 1f_seed_submission_status_types.py
│   ├── 30a_seed_field_types.py
│   ├── 33a_seed_notification_types.py
│   ├── 33b_seed_comment_types.py
│   └── __init__.py
```

i Nota: Nota: Cualquier script de inicialización nuevo que se añada a la carpeta `seeds/` debe implementar obligatoriamente la función `upgrade()`. Se recomienda seguir el patrón numérico/alfabético prefijado (`00a_` , `10a_`) para controlar de forma explícita el orden de carga y evitar fallos por restricciones de llave foránea en la base de datos.

6.4 Poblado de históricos (populator)

Este componente gestiona la ingesta masiva de datos históricos y estructuras complejas en el sistema a través de la API pública de los microservicios, en lugar de realizar inserciones directas en el motor de persistencia. El módulo se orquesta de manera asíncrona mediante `asyncio` y `httpx`, conectándose con el servicio de autenticación (`api_auth`) y el núcleo del sistema (`api_core`) utilizando variables de entorno para resolver los endpoints internos de la red de Docker. El flujo extrae las credenciales iniciales de un archivo TOML administrado mediante secretos de Docker (`/run/secrets/users_file`), priorizando cuentas de nivel `root` para autenticarse, obtener el token Bearer correspondiente e inyectarlo automáticamente en las cabeceras de las peticiones. Para mitigar fallas en tareas de larga duración, la capa del cliente (`ServiceClient`) implementa mecanismos de re-autenticación automática que refrescan el token de acceso si este expira a mitad de una transacción.

La extracción de la data histórica se delega a un conector dedicado (`GitHubConnector`) que descarga los conjuntos de datos crudos o estructuras estructuradas directamente desde repositorios remotos utilizando un token de acceso seguro provisto como secreto de Docker (`/run/secrets/github_token_seeds`). Cada registro recuperado pasa por una capa estricta de validación local mediante esquemas de Pydantic antes de despacharse hacia la API pública de `api_core`. La transferencia de datos se realiza tanto en registros individuales como en procesamiento por lotes que comparten el mismo grupo de conexiones para optimizar el rendimiento. Al procesar las solicitudes mediante peticiones HTTP estándar (`POST`), el sistema garantiza que toda la lógica de negocio, validaciones relacionales y disparadores de eventos del backend se apliquen correctamente a la data histórica, tal como ocurriría con la interacción regular de un usuario.

La estructura física del módulo organiza las tareas en el directorio `pops/` de forma secuencial, incluyendo scripts específicos de migración y plantillas locales en formatos estructurados (CSV y XLSX) dentro de subdirectorios de trabajo:

```
Persistence/src/populator
├── pops
│   ├── 10a_seed_sectors.py
│   ├── 10b_seed_entities.py
│   ├── 11a_seed_section_types.py
│   ├── 11b_seed_forms.py
│   ├── 11c_seed_sections.py
│   ├── 11d_seed_questions.py
│   ├── 11e_seed_loop_questions.py
│   ├── 11f_seed_card_templates.py
│   ├── 11g_seed_field_groups.py
│   ├── 11h_seed_fields.py
│   ├── 11i_seed_field_choices.py
│   ├── 12a_seed_field_dependencies.py
│   ├── 12b_seed_field_rules.py
│   └── 13a_seed_grading_criteria.py
```

```

├── __init__.py
├── jhonatan
│   ├── actors.actor_segments_template.csv
│   ├── actors.actors_template.csv
│   ├── Entidades.csv
│   ├── Estructura_IIP.xlsx
└── populator.py

```

i Nota: Nota: A diferencia de los módulos de esquemas, migraciones y datos maestros, el proceso de población masiva histórica no se ejecuta de forma mandatoria con la bandera `--setup` en sistemas limpios si los secretos o conectores externos de GitHub no están mapeados. Este componente requiere que tanto el contenedor de autenticación como el servicio núcleo estén completamente activos, saludables y con la base de datos ya estructurada.

6.5 Backup y Restore

Este sistema gestiona los respaldos de la base de datos PostgreSQL mediante un volumen en Docker Compose que vincula el directorio local `./Persistence/src/backups` con la ruta interna `/backups` del servicio `persister`.

```

persister:
  ...
  volumes:
    - ".:/Persistence/src/backups:/backups"

```

La administración se automatiza a través del script `launcher.sh` empleando dos banderas principales: `--backup`, que ejecuta de forma no interactiva un `pg_dump` nombrando el archivo con una marca de tiempo para evitar sobrescrituras, y `--restore`, que requiere el nombre del archivo `.sql` como argumento e inyecta la variable `PGPASSWORD` internamente para autenticar de manera automática la restauración mediante `psql`.

Dado que el proceso de restauración sobrescribe los datos actuales de la base de datos, se recomienda ejecutar un respaldo preventivo antes de realizar cualquier importación.

i Nota: Nota: Se debe asegurar que el directorio local `./Persistence/src/backups` cuente con los permisos de lectura y escritura correctos para que el contenedor pueda almacenar los archivos.

La gestión de respaldos se centraliza en el script de automatización utilizando las siguientes banderas:

6.5.1 Crear un Respaldo (`--backup`)

Genera una copia de seguridad en caliente de la base de datos PostgreSQL utilizando la herramienta `pg_dump`. El comando se ejecuta de forma no interactiva (`-T`) y almacena el archivo resultante usando una marca de tiempo

(`TIMESTAMP`) para evitar la sobrescritura de respaldos anteriores.

```

--backup)
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
BACKUP_DIR="./Persistence/src/backups"

mkdir -p "$BACKUP_DIR"

echo "Creating database backup inside
$BACKUP_DIR..."

if docker compose exec -T db sh -c "pg_dump -
U \"${USER_VAR}\" -d \"${DB_VAR}\" > /backups/
db_backup_${TIMESTAMP}.sql"; then
    echo "✅ Backup completed successfully:
$BACKUP_DIR/db_backup_${TIMESTAMP}.sql"
else
    echo "❌ Backup failed."
fi
;;

```

6.5.2 Restaurar un Respaldo (`--restore`)

Permite restaurar un archivo `.sql` específico cargándolo directamente en el motor de la base de datos mediante `psql`. El script requiere el nombre del archivo como parámetro e inyecta la variable `PGPASSWORD` de forma interna para realizar la autenticación automática sin solicitar credenciales en la terminal.

```

--restore)
INPUT_FILE="${1:-}"
if [ -z "$INPUT_FILE" ]; then
    echo "Usage: ./launcher.sh --restore
<backup_filename.sql or path>"
    echo "Example: ./launcher.sh --restore
db_backup_20260618_232418.sql"
    exit 1
fi

FILE_NAME=$(basename "$INPUT_FILE")

echo "Restoring database from $FILE_NAME..."

# Inject PGPASSWORD so Postgres authenticates
automatically without prompting
if docker compose exec -T db sh -c
"PGPASSWORD=\"\${POSTGRES_PASSWORD}\" psql -
h db -U \"${USER_VAR}\" -d \"${DB_VAR}\" < /
backups/$FILE_NAME"; then
    echo "✅ Restore completed successfully."
else
    echo "❌ Restore failed. Make sure $FILE_NAME
exists in your backups directory."
fi
;;

```


7 autenticación y Autorización

- 7.1 Servicio de autenticación**
- 7.2 autenticación JWT**
- 7.3 Roles y permisos**
- 7.4 Sesiones de refresco**
- 7.5 Seguridad de contraseñas**

8 Arquitectura de API

8.1 Flujo de Datos

El flujo de información es transaccional y validado. Cuando un cliente solicita una operación, el tráfico es enrutado por Nginx hacia el servicio correspondiente tras validar el contexto de seguridad. La secuencia operativa típica, donde el servicio Core interactúa con PostgreSQL mediante los repositorios DDD, se describe en Figura 5. Este diseño asegura que cada petición sea trazable, consistente y eficiente, permitiendo que la plataforma soporte tanto la carga administrativa actual como las futuras demandas de analítica avanzada.

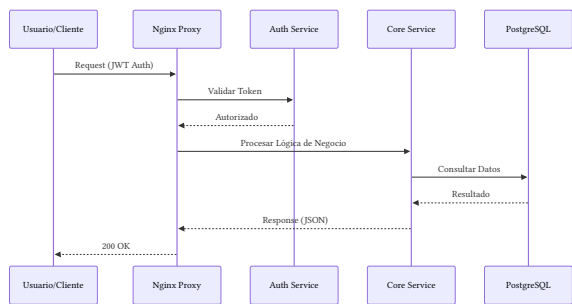


Figura 5: Diagrama de secuencia del flujo de una petición API.

9 Configuración y Despliegue

- 9.1 Requisitos previos**
- 9.2 Configuración de Docker Compose**
- 9.3 Gestión de variables de entorno y secretos**

10 Arquitectura de Datos y Persistencia

- 10.1 Diseño del modelo relacional**
- 10.2 Estrategia de migraciones**
- 10.3 Separación por esquemas**
- 10.4 Seedeo de iniciales**
- 10.5 Seedeo de históricos**

11 API y Endpoints

- 11.1 Documentación de la API**
- 11.2 Estándares de comunicación y seguridad**

12 Desarrollo y Utilidades

- 12.1 Estructura de directorios del mono-repo**
- 12.2 Uso de utilidades Python internas.**
- 12.3 Guía de contribución y estilo de código.**

13 Hoja de Ruta (Roadmap) y Futuras Integraciones

- 13.1 Implementación de Celery Workers para procesos en segundo plano.**
- 13.2 Integración de sistemas de Vectorización y RAG.**
- 13.3 Implementación de servidores MCP para agentes de IA.**

14 Apéndice y Glosario

15 Eraser

1. Introduction 1.1 Purpose 1.2 Scope 1.3 Intended Audience 1.4 Platform Overview 1.5 Design Principles
2. System Architecture 2.1 High-Level Architecture 2.2 Architectural Decisions 2.3 Service Overview 2.4 Monorepo Structure 2.5 Technology Stack
3. Deployment Architecture 3.1 Infrastructure Overview 3.2 Docker Architecture 3.3 Reverse Proxy (Nginx) 3.4 Service Communication 3.5 Configuration Management 3.6 Environment Variables 3.7 Secrets Management
4. Domain Architecture 4.1 Domain-Driven Design 4.2 Layered Architecture 4.3 Application Services 4.4 Repositories 4.5 Unit of Work 4.6 Shared Library 4.7 Request Lifecycle
5. Data Architecture 5.1 Database Overview 5.2 Schema Organization 5.3 Domain Entities 5.4 Relationships 5.5 Historical IIP Versions 5.6 Data Integrity 5.7 Auditability
6. Persistence 6.1 SQLAlchemy 6.2 Alembic Migrations 6.3 Seeders 6.4 Historical Data Population 6.5 Backup and Recovery
7. Authentication & Authorization 7.1 Authentication Service 7.2 JWT Authentication 7.3 Roles & Permissions 7.4 Refresh Sessions 7.5 Password Security
8. API Design 8.1 REST Conventions 8.2 API Structure 8.3 Authentication Flow 8.4 Error Handling 8.5 Pagination & Filtering 8.6 OpenAPI Documentation
9. Development Guide 9.1 Repository Structure 9.2 Workspace Management 9.3 Dependency Management 9.4 Local Development 9.5 Testing 9.6 Code Style 9.7 Contributing
10. Operations 10.1 Deployment 10.2 Logging 10.3 Monitoring 10.4 Performance 10.5 Maintenance
11. Future Evolution 11.1 Background Workers 11.2 Open Data Integration 11.3 Semantic Search 11.4 Vector Database Integration 11.5 Retrieval-Augmented Generation (RAG) 11.6 MCP Integration
12. Appendix 12.1 Directory Structure 12.2 Configuration Reference 12.3 Glossary 12.4 Acronyms

Bibliografía

- [1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2004.